H2S-NN Revision: Cumulative Progress Report

The progress reports of the revision in one document, one chapter per report, each opened by a page with its date

8–9 November 2026

Chapter	Date	Content
1	8 November 2026	**Scalability and path coupling** Lighter restriction maps, meta-path faces coupled directly to nodes, fact storage, path sampling and computation, node gates, cost
2	8 November 2026	**Encoder and geometry** Data-pipeline check, frequency-selective filter, neighbourhood summary, known-answer test of the path mechanism, cost
3	9 November 2026	**Heterophily of the evaluation datasets** The two heterophily measures, a check against a published value, one table for all node-classification datasets

CHAPTER 1

8 November 2026

Scalability and path coupling

Lighter restriction maps, meta-path faces coupled directly to nodes, fact storage, path sampling and computation, node gates, cost

H2S-NN Revision: Progress Report

Making the model scalable, letting path information reach the nodes, and fixing the data pipeline

8 November 2026 Measurements: ACM dataset, NVIDIA RTX 4060 Laptop GPU (8 GB)

Overview

The reviewers' main concerns included whether H2S-NN scales to larger graphs. While preparing the revision, we also found a structural problem: in the submitted model, the term that was meant to carry information from the meta-path instances (the 2-cells) to the nodes had exactly zero effect during diffusion. Two main changes address these, followed by three supporting changes (Section 3):

1. **Lighter restriction maps.** Each edge no longer stores its own full `d×d` matrix.
2. **Meta-path faces coupled directly to nodes.** Each meta-path instance is still a face (2-cell) of the model. It now passes information straight to every node on it, through maps that the face itself determines, and the edge-level machinery of the old model is removed.

15.9×	**5.5×**	**4.9×**	**5.3×**
less memory for the restriction maps	less GPU memory for the whole model	faster per training step	fewer parameters (1.62M → 0.31M)

Whole-model memory and speed are compared at stalk dimension d = 8, the largest size at which the old model still fits on our 8 GB GPU. The map memory and parameter counts are for d = 32, the setting used in the paper. At d = 32 the old model does not fit on the GPU at all, while the new model needs 2.5 GB.

1. Lighter restriction maps

Before

For every edge, the model learned two restriction maps, one for each endpoint. Each map was a full `d×d` matrix. On ACM there are 547,872 edges, so at d = 32 the maps alone need about 1.1 billion numbers (4.2 GB). This is the main reason the submitted model could not run on large heterophilic benchmarks such as ogbn-mag.

After

We split each map into three parts, and only the small parts are stored per edge:

```
map for node v on edge e  =  c_e  ×  R_relation  ×  (k reflections built from node features)
```

- **A rotation per relation type** (R): for example, all paper→author edges share one. ACM needs only 16 such matrices in total, instead of two per edge.
- **k reflections per edge**: these make the map depend on the actual nodes. A reflection is a `d×d` matrix, but it is fully described by a single vector of d numbers. It can be applied to a feature vector directly ($x - 2u(u^\top x)/\|u\|^2$), so the matrix is never built. We use k = 2.
- **One scale per edge** (c_e): a single positive number, shared by both endpoints. Sharing it keeps the transport along a path a pure rotation, so long paths cannot blow up numerically.



Figure 1. What is stored for each edge at d = 32. The large matrices become shared per relation type; each edge keeps only a few vectors.

Choosing k

k sets the trade-off between flexibility and memory. With k = d reflections, the map could be any rotation, which is as expressive as the old model but saves nothing. With a small k, edges of the same relation share the rotation and differ only in a few directions.

Restriction maps on ACM, d = 32	Numbers stored	Reduction
Before: full matrices	1,122,041,856	—
After, k = 1	35,628,064	31.5×
After, k = 2 (chosen)	**70,691,872**	**15.9×**
After, k = 4	140,819,488	8.0×

Result of this change alone

At d = 8, whole-model GPU memory halved (6.08 → 3.00 GB) and a training step became 1.8× faster (3.18 → 1.73 s). The part of the computation that builds and applies the maps became 5× faster. Total memory did not fall by the full 15.9×, because the rest of the old model (edge states and the dense maps on the path side) still dominated. The second change removes that part.

2. Meta-path faces coupled directly to nodes

What a meta-path face means

Each meta-path instance, for example P_1–A–P_2 (two papers with a common author), is a face (2-cell). Its boundary is the instance itself together with a **closure edge** P_1–P_2, which stands for the meta-path relation "P_1 and P_2 are related through Paper–Author–Paper". The face states a coherence condition: going from P_1 to P_2 along the meta-path relation must agree with going through the instance. In sheaf terms, this condition forces the map of the closure edge to equal the composition of the maps along the instance (the path's *holonomy*). The meta-path relation is therefore not a new relation learned freely, as in HAN, but is fixed by the instance that witnesses it, including its intermediate node and the order of its relations. This meaning is the same before and after the change; what changes is how the face reaches the nodes.

Before

The face had its own term, but that term influenced the nodes only indirectly: face → states stored on edges → nodes. This route has a mathematical problem. When the diffusion is linear, which it is in our model, the face term's influence on the nodes is **exactly zero**. This holds for any number of diffusion steps, and we also confirmed it numerically. Path information still reached the prediction through two other routes:

- the closure edge P_1–P_2, used as an ordinary edge of the graph operator. Its map was the holonomy, as the face requires, but the face's own term added nothing beyond this edge;
- a single average of the edge states in the final classifier.

In addition, the edge route required storing a large tensor on every edge and every path, which used most of the GPU memory.

After

The face is now coupled directly to **every** node on its boundary path: both endpoints and the intermediate node(s). The coupling uses the face's own restriction maps. These are not new parameters: the face's coherence condition forces them to be the transports along the path, `T_i = Q₁Q₂···Q_i`, the same maps that fix the closure edge. Each diffusion step does two things:

1. It forms the face's value `z_f`: the average of its nodes' features after transporting them into the face, so that relation order and direction matter. `z_f` lives in the face's own space.
2. It pulls every node on the path toward `z_f` (transported back to that node), together with the usual pull toward its graph neighbours.

Both pulls together form a single **sheaf Laplacian**. The cellular sheaf on the 2-complex (nodes, edges, closure edges and faces, with their maps) is kept as it was. What is removed is the computation on edges: the edge states, the edge Laplacian and the dense path-side matrices.

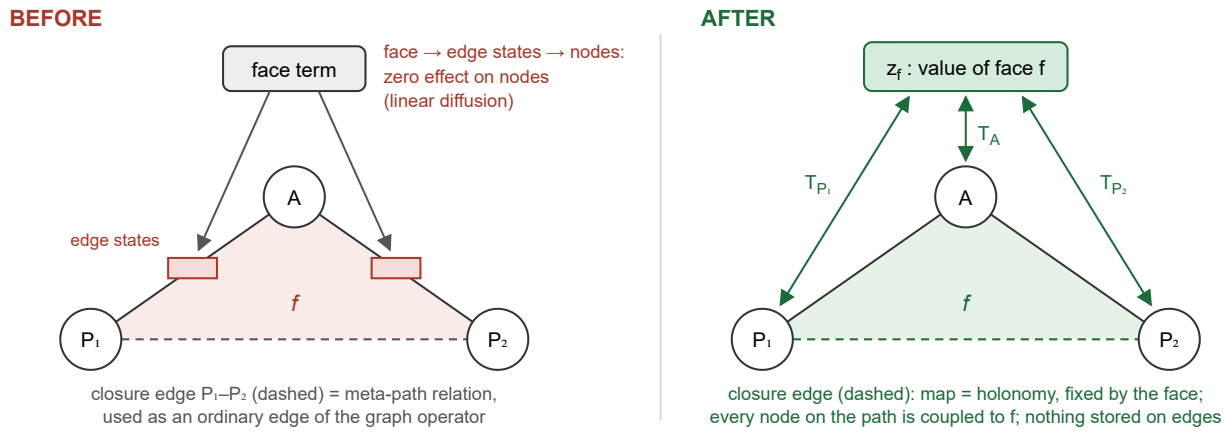


Figure 2. How a meta-path face (Paper–Author–Paper) influences the nodes, before and after. In both, the face f is bounded by the instance P_1–A–P_2 and the closure edge P_1–P_2, whose map the face fixes to the holonomy of the path. After the change, the face is coupled to every node on the path through the maps T, which the face also fixes; information flows both ways.

Properties we proved and checked numerically

- **It respects the sheaf structure.** The new term never contradicts the graph's own sheaf: any assignment that is consistent on the graph stays consistent. If the maps on the path were chosen freely, as in a hypergraph model, this property would be lost. We checked both cases.
- **It is stable.** After normalization, the operator's largest eigenvalue is at most 2, so the diffusion step cannot diverge for any learned step size. The largest value observed over 60 random test graphs was 1.99999. Without normalization, values reached 10^{16}.
- **The face reaches the nodes.** In one step, the face carries information between all nodes on the path. In the old model the face term had zero effect on the nodes.
- **The intermediate node shapes the meta-path relation.** The holonomy from P_1 to P_2 is the composition of the maps of both boundary edges, and these depend on the intermediate node's features. Two Paper–Author–Paper

instances with different authors therefore give different relations between their papers. (The intermediate node also reaches the endpoints in one step through the ordinary edges; that part is not new.)

What this change gives up, and what it keeps

The face term can be rewritten exactly as pairwise terms between all nodes on the path (n = number of nodes on the path):

```
Σ_i ‖T_i x_i − z_f‖²  =  (1/n) Σ_{i<j} ‖T_i x_i − T_j x_j‖²
```

For P_1–A–P_2 the three pairs are P_1–A and A–P_2, with exactly the maps of the existing edges, and P_1–P_2, with the holonomy map, that is, the closure edge. So the face reaches the nodes through the meta-path relation it defines and by strengthening the edges of its own instance, each pair with weight `γ·w_f/3`. This is what a meta-path face is meant to do. The difference from HAN-style meta-path edges is therefore not that the endpoint relation is absent; it is that the relation's map is fixed by the instance instead of being learned freely. Several consequences follow, and the paper will state them:

- **Values on edges and faces are no longer evolved.** The sheaf on the 2-complex is unchanged, but the model no longer runs the Hodge Laplacian on edge and face values. In the submitted model that part never reached the nodes (see Before), so no effect on the nodes is lost. What is lost is the option to read edge or face values out directly.
- **The face is a definition, not a measurement.** The closure edge is not observed in the data; its map is defined by the face, so every face is consistent by construction. A face cannot detect conflicting relations. It imposes that the meta-path relation agrees with its instance.
- **The same operator could be written without faces**, as transports along paths. The face is what justifies this choice: its coherence condition allows only the path transport as the map of the meta-path relation, and a free choice breaks the sheaf property listed above.
- **The meta-path relation may now be weaker.** In the submitted model the closure edge was a full edge of the graph operator; now it carries weight `γ·w_f/n`. Whether this helps or hurts accuracy has not been measured. The ablations in Section 5 compare the two directly.

Result

Stalk dimension d	Old model	After change 1	After both changes
32 (paper setting)	**does not fit in 8 GB**	**does not fit in 8 GB**	**2.5 GB, 1.8 s**
16	does not fit in 8 GB	6.6 GB, 6.0 s	1.4 GB, 1.0 s
8	6.1 GB, 3.2 s	3.0 GB, 1.7 s	1.1 GB, 0.65 s
Parameters at d = 32	1.62M	0.46M	0.31M

Peak GPU memory and time for one training step (forward and backward) on ACM, averaged over 5 runs.

At d = 8, the new model uses 5.5× less memory and is 4.9× faster than the old one. At d = 32, the setting in the paper, the old model no longer fits on our GPU, while the new one needs 2.5 GB. A 20-epoch training run was stable throughout.

3. Further changes: graph encoding, path sampling and node gates

Once faces were coupled directly to nodes, three more parts of the pipeline needed attention: how the graph's edges are represented, how path instances are sampled and computed, and whether influence between nodes has to be symmetric.

3.1 Each fact is stored once

Before

The HGB benchmarks store every relation in both directions. In ACM, for example, the relation *paper* → *author* (PA) has an exact mirror *author* → *paper* (AP) that contains the same pairs reversed. All 10 relation pairs in ACM, DBLP and IMDB are exact mirrors of this kind. Standard message-passing GNNs need both directions, because messages flow from source to target only. The old model inherited this and treated the single fact "paper p is written by author a" as two separate edges, each with its own unrelated maps.

Why it is a problem

The sheaf Laplacian is already symmetric, so the second copy adds no direction, only a second, conflicting constraint. For one fact, the energy has two terms:

```
E(x) = ‖F¹_src x_p − F¹_tgt x_a‖²  +  ‖F²_src x_a − F²_tgt x_p‖²
```

A consistent assignment ($E = 0$) must satisfy $x_a = Q_1 x_p$ and $x_a = Q_2 x_p$ at the same time. With two independently learned maps, $Q_1 \neq Q_2$, and generically only $x = 0$ survives. On a single link at $d = 8$, the space of consistent assignments collapsed from 8 dimensions to 0, which makes the theory's guarantee that "path cells never contradict the graph sheaf" empty.

After

Each fact is now one edge. Two relations are recognised as inverses by one exact, parameter-free rule:

```
ρ(r) = r'   if and only if   the pairs of r' are exactly the pairs of r reversed
a fact:  (u, r, v) ≡ (v, ρ(r), u)   →   stored once, as one oriented edge
```

Direction is kept: when a path walks along an edge against its stored orientation, it uses the reverse transport $Q^\top$ instead of Q. If two relations are not exact mirrors, they stay separate.

Result

- ACM goes from 547,872 stored edges to 273,936 facts, so the graph operator is half the size.
- **We found a data leak in link prediction.** When a link (p, a) was held out for testing, the old split removed it from PA but left its mirror (a, p) in AP, so the model could see the answer. On a test case, 8 of 21 held-out links leaked. Splits are now made per fact. Earlier link-prediction numbers, ours and the baselines', may be inflated on datasets with mirror relations and will be rerun.
- **We also found a loader bug in MUTAG.** Bond types were assigned by position after PyTorch Geometric had reordered the edges, so 1,771 of 7,442 bonds had the wrong type, in all 188 graphs. All earlier MUTAG results will be rerun.
- Baselines still receive the standard stored graph with both directions, so every method sees the same facts.

3.2 Path instances: unbiased sampling and faster computation

Before

Path instances were collected by depth-first search with a fixed quota of r paths per starting node. This has three problems.

- **It is biased.** Take a paper with 3 authors, each with 30 papers, and r = 20. The search fills the quota through the first author and never reaches the other two. In 200 of 200 random seeds, only 1 of the 3 authors was used as the middle node. With one author who has 300 papers, 48–50 of 50 paths went through that author. The question "does the middle node matter?", which the reviewers raised, cannot be answered if the sampler always picks the same middle node.
- **Duplicates are counted twice.** The path p_1–a–p_2 and its reverse p_2–a–p_1 became two cells (3,690 of 16,300 in a test), so their weight depended on chance.
- **It is slow**: it is a Python loop per starting node, and was the preprocessing bottleneck for larger graphs.

After: sampling

Paths are drawn by random walks that follow the meta-path's relation types and directions. Each step picks uniformly among the allowed neighbours, so every path has a known probability:

```
p(path | start a, meta-path π)  =  ∏ over steps j  of  1 / (number of allowed neighbours at
step j)
r walks per (start node, meta-path), each with weight 1/r
identical paths (in either direction) are merged by ADDING their weights:  m_f = Σ weights
```

Adding rather than discarding duplicates makes the sampled operator an unbiased estimate of its expected value: on a test, the error was 0.005 with summing versus 0.47 with discarding. In the example above, each of the 3 authors is now chosen with probability 1/3. Paths are resampled every training epoch; validation and test use one fixed draw.

After: computation

For a path v_0, …, v_L with step transports Q_1, …, Q_L, the transport of node i into the path is $T_i = Q_1 Q_2 \cdots Q_i$. The old code built each T_i separately: L(L+1) transport applications per path. Because each T_i is a scaled rotation ($T_i^\top T_i = s_i^2 I$), the same result follows from two passes of L steps each:

```
path term at node i:   w_f · ( s_i² x_i − T_iᵀ z_f ),        z_f = (1/n) Σ_i T_i x_i
forward pass:    Σ_i T_i x_i = x₀ + Q₁( x₁ + Q₂( x₂ + ⋯ + Q_L x_L ) )      (L steps)
backward pass:   T₀ᵀz = z,   T_iᵀz = Q_iᵀ ( T_{i−1}ᵀ z )                  (L steps)
```

A Paper–Author–Paper path (L = 2) needs 4 transport applications instead of 6, and a path of length 4 needs 8 instead of 20. The output is identical (difference $\leq 2 \cdot 10^{-12}$ in double precision).

Result of 3.1 and 3.2 together

On ACM at d = 32, a training step now uses **1.49 GB and 1.06 s**, down from 2.52 GB and 1.77 s after Section 2, on the same GPU. This figure includes the gates of Section 3.3. The two measurements also differ in the number of sampled paths, so the reduction should not be attributed to one change alone.

3.3 Listen and broadcast gates (proposed, under test)

Before

The diffusion $x \leftarrow x - \eta\, \hat{\Delta} x$ is symmetric: node u pulls node v exactly as hard as v pulls u.

Why we are considering a change

In heterogeneous graphs, influence is plausibly unequal. In ACM, one subject node is linked to as many as 1,195 papers. It should be able to inform its papers without being pulled toward each of them. Under heterophily, a node should also be able to stop listening to dissimilar neighbours. This is a hypothesis, not yet an observed effect.

Proposal

Each node gets a *listen* gate g and a *broadcast* gate b, both between 0 and 1, computed from its input features h_0:

$$g_v = \sigma(a_g^\top h_{0,v} + \beta_g), \qquad b_v = \sigma(a_b^\top h_{0,v} + \beta_b)$$
$$x \leftarrow x - \eta \cdot G\, \hat{\Delta} B\, x, \qquad G = \mathrm{diag}(g),\ B = \mathrm{diag}(b)$$
$$\text{influence of } v \text{ on } u \;=\; g_u \cdot b_v \qquad \text{(no longer symmetric)}$$

Example: if a subject node learns $b \approx 1$ and $g \approx 0.1$, it informs its papers fully but is pulled only weakly by them.

Non-symmetric operators usually have complex eigenvalues and can be unstable. This one cannot, because it is similar to a symmetric matrix:

$$\text{with } S = (G^{-1}B)^{1/2}: \quad S\,(G\,\hat{\Delta} B)\,S^{-1} \;=\; (GB)^{1/2}\,\hat{\Delta}\,(GB)^{1/2} \qquad \text{(symmetric, positive semidefinite)}$$
$$\Rightarrow \text{all eigenvalues are real and lie in } [0,\ 2\cdot\max g\cdot b) \subset [0,\ 2) \;\Rightarrow\; \text{the step is stable for any } \eta < 1$$

On 200 random test graphs, the largest imaginary part of any eigenvalue was $7 \cdot 10^{-17}$. The comparable operator of Cooperative Sheaf Neural Networks reached $2.6 \cdot 10^{4}$ on the same graphs. The gates add 0.6% to the time and 0.5% to the memory of a training step.

Status. We do not yet know whether the gates improve accuracy. The gates are currently part of the model. They will be compared against the symmetric model (all gates equal to 1) over 10 seeds with paired tests, and that comparison will be reported in the paper.

4. Cost in big-O terms

	Old model	New model
Memory for restriction maps	$O(M \cdot d^2)$	$O(M \cdot k \cdot d + \lvert R \rvert \cdot d^2)$
Memory on the path side	$O(F \cdot L \cdot d^2)$	nothing stored (computed on the fly)
Time per diffusion step	at least $O((M + F \cdot L) \cdot d^2)$	$O((M + F \cdot L) \cdot d^2)$
Transport applications per path	$L(L+1)$	$2L$
Number of edges M on ACM	547,872	273,936 (each fact once)

M = number of edges, F = number of path instances, L = path length, |R| = number of relation types, k = 2.

The asymptotic saving is in **memory**: from d^2 numbers per edge to about $k \cdot d$. Time per step keeps the same order, because applying a rotation to a vector still costs d^2. The measured speed-up (up to 4.9×) comes from no longer

generating and reading a full matrix for every edge, and from grouping edges by relation type so the shared rotations are applied in large batches.

CHAPTER 2

8 November 2026

Encoder and geometry

Data-pipeline check, frequency-selective filter, neighbourhood summary, known-answer test of the path mechanism, cost

H2S-NN Revision: Progress Report

Letting the model keep disagreement, giving it a view of each node's neighbourhood, and testing the path mechanism directly

8 November 2026 (second report) Measurements: ACM dataset, NVIDIA RTX 4060 Laptop GPU (8 GB)

Overview

The previous report made the model fit in memory and coupled each meta-path face directly to every node on it. This stage asked what the model does with that information once it reaches a node, and whether the path mechanism does what the thesis claims. Two changes and two checks came out of it:

1. **A frequency-selective encoder.** The diffusion step could only smooth. A learned filter now lets each feature channel keep, damp or sharpen the disagreement between neighbours.
2. **Geometry that sees the neighbourhood.** The restriction maps, gates and path weights now read a summary of each node's typed neighbours, not only the node's own features. The submitted paper describes this term, but the code did not implement it.

Two checks support these changes (Section 3): the data pipeline against the official benchmark, and a synthetic test of the path mechanism in which we know which path instance holds the answer.

+0.3%	**+39%**	**Exact**	**1.0 vs 0.54**
time for the new filter (memory +0.04%)	time for the neighbourhood summary	match of our splits, labels and edges with the official benchmark	path test accuracy with correct vs learned path weights (chance 0.5)

No benchmark accuracy is reported. All runs so far use untuned default settings. Accuracy comparisons will run on the cluster under one protocol, with the same tuning budget for every method.

1. A frequency-selective encoder

Before

Propagation was K small diffusion steps. Each step pulled every node toward its neighbours, after transporting the neighbours with the restriction maps:

```
x  ←  (I − η A)^K h_0          A = gated, normalised sheaf Laplacian of edges and path faces
```

A signal on the graph can be read as a mix of frequencies. A low-frequency signal is one where each node agrees with its transported neighbours. A high-frequency signal is one where it disagrees. The operator A measures this, with eigenvalues from 0 (full agreement) to 2 (strong disagreement). Each step multiplies a component at frequency λ by $(1 - \eta\lambda)$, so K steps multiply it by $(1 - \eta\lambda)^K$.

Why it is a problem

The old update can only smooth, and it smooths every feature channel in the same way. With the step size the model learned ($\eta \approx 0.45$) and $K = 4$:

- a component at $\lambda = 1$ keeps about 9% of its size;

- a component at $\lambda = 1.5$ keeps about 1%.

Under heterophily, the disagreement itself can carry the label. A paper whose subject is shared mostly by papers on a different topic is informative *because* it differs from them. The old update erases exactly this signal.

After

The output is a learned polynomial filter of the same operator:

$$x = \Sigma_{k=0..K} T_k(A - I) \cdot W_k h_0 \qquad \text{with} \quad \Sigma_k \|W_k\| \le 1$$

- T_k are the Chebyshev polynomials, a standard, numerically well-behaved basis.
- W_k are small d×d matrices, one per degree, that mix the feature channels.

Each channel therefore gets its own frequency response (Figure 1). The filter applies A K times, as the old K steps did, and adds only small per-node matrix products, so the memory savings of the previous report are kept.

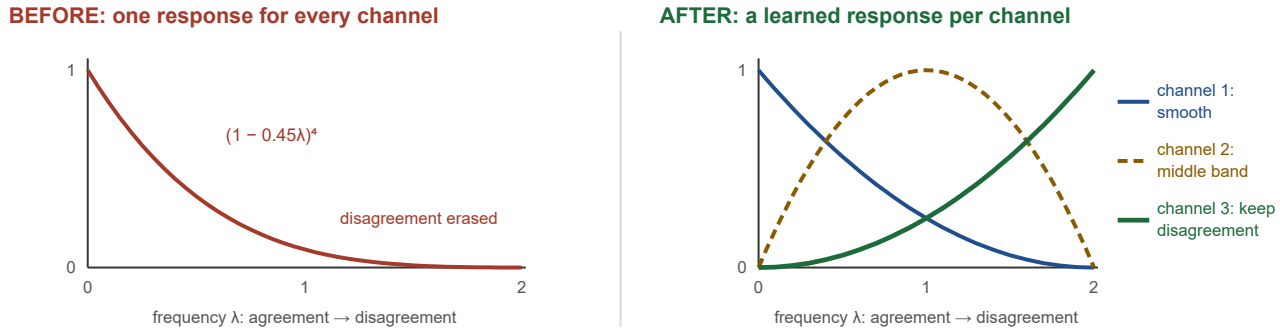


Figure 1. How strongly each frequency survives propagation. Before: one fixed curve for all channels, shown with the learned $\eta \approx 0.45$ and $K = 4$. After: each channel has its own curve. The three curves on the right are example shapes the filter can represent, not learned values.

Example. One channel can carry a paper's topic with a smoothing response, so papers that share authors move toward agreement. A second channel can use a high-pass response. A paper whose topic differs from the other papers on its subject then keeps that difference, instead of losing 99% of it as in the old update.

Properties we proved and checked numerically

- **It is stable for every parameter value.** The gated operator has a real spectrum in [0, 2), as the previous report showed. With the bound on the W_k, the filter's gain is at most 1, whatever values are learned.
- **The old update is a special case.** Setting every W_k to a fixed multiple of the identity gives $(I - \eta A)^K$ exactly. At initialisation the new model computes the same output as the old one, to within 10^{-10}.
- **The listen and broadcast gates keep their meaning through the whole filter.** A node whose listen gate is closed receives nothing from any other node, and a node whose broadcast gate is closed sends nothing. The corresponding blocks are exactly zero.
- **The node's own signal is never washed out.** The degree-0 term keeps the input, so adding degrees cannot make the output vanish. Deep sheaf diffusion models are known to suffer from that problem.

What this change gives up, and what it keeps

- **The energy reading is lost.** The old update could be read as a flow that lowers the sheaf energy at every step. The filter can favour disagreement, so this reading no longer holds and the paper will not claim it. Stability and the real spectrum are kept.

- **The stability bound limits what can be learned.** In a short smoke run, the unconstrained filter weights exceeded the bound within five epochs (sum 1.067) and were scaled back. Training pushes against the guarantee. That is the price of a guarantee that holds for every parameter value, and we will measure it rather than assume it is free.

Result

ACM, d = 32, one training step	Peak GPU memory	Time	Parameters
Old update	1.57 GB	1.47 s	532,649
Frequency-selective filter	**1.57 GB**	**1.48 s**	**537,768**

Both with the neighbourhood summary of Section 2. Mean of 3 runs after a warm-up. The cost limit set in advance was 25% time and 10% memory; the measured cost is 0.3% and 0.04%.

Status. That frequency selectivity helps under heterophily is a hypothesis we have not yet observed. On the cluster the filter will be compared with the old update, which is nested inside it, on the heterophilic datasets (Freebase, IMDB and the H2GB sets).

2. Geometry that sees the neighbourhood

Before

The restriction maps, the listen and broadcast gates and the path weights were all computed from a node's own input features h_0. The submitted paper includes a neighbourhood-context term for the maps, but the code did not implement it, so paper and code disagreed.

Why it is a problem

Two nodes with similar features can play different roles in the graph. If the geometry sees only the node itself, it cannot tell them apart, and both receive the same maps and gates.

After

Each node gets a typed neighbourhood summary: for each relation type, the average input features of its neighbours through that relation. The geometry reads both parts:

```
c_v  =  [ mean of h_0 over v's authors ,  mean over v's subjects ,  … ]        (one average per relation type)
maps, gates, path weights  =  functions of  [ h_0v , c_v ]
```

The summary is computed **once**, before propagation. Other sheaf models (NSD, CSNN, DNSD) recompute their maps from the propagated states at every layer, which creates a new sheaf at each layer. Computing the summary once keeps a single fixed sheaf. The guarantees of Section 1 and the mini-batch argument of the previous report therefore still hold.

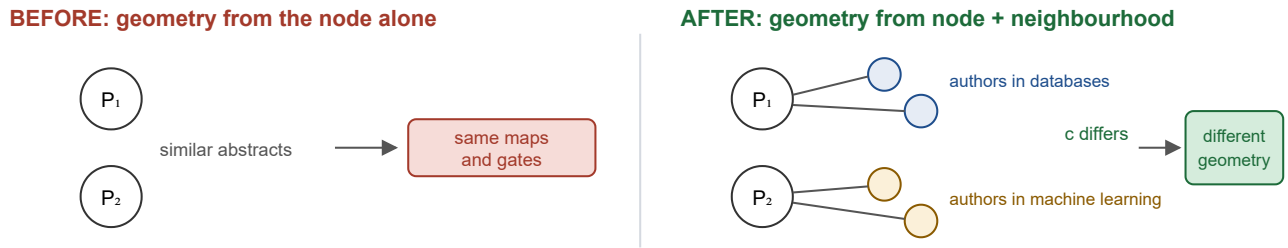


Figure 2. Two papers with similar abstracts. Before, they receive the same maps and gates. After, the summary of their authors separates them.

Example. P_1 and P_2 have nearly identical abstracts. P_1's authors publish mostly in databases and P_2's mostly in machine learning, so their summaries differ. P_2 can then learn, for instance, to listen less to a subject node that is shared mainly with database papers.

Properties we proved and checked numerically

- **Nothing else changes when the summary is switched off.** The model then reproduces the previous model's outputs exactly.
- **The summary reaches the geometry, and only where it should.** Changing one neighbour's features changes that node's maps and gates when the summary is on, and changes nothing when it is off. The summary uses only the stored facts and respects each relation's direction.

What this change gives up, and what it keeps

- **The geometry does not follow the propagated states.** It is fixed before propagation, by design: this keeps one sheaf and all its guarantees. Models that recompute their maps at every layer can adapt their geometry during propagation; ours cannot.
- **It is the most expensive change of this stage** (Result, below).
- **It overlaps partly with the path faces.** The summary gives each node a view two hops away: an author's summary contains its papers, and a paper reads that summary through its author. Section 3.2 shows that this alone solved a task we had designed to test path faces. This is not a defect, but it raises the bar for the thesis. Path faces must add information beyond edges *plus* neighbourhood summaries. We therefore keep the summary on in every arm of the path comparison, which is the stricter test, and the paper will say why.

Result

ACM, d = 32, one training step	Peak GPU memory	Time	Parameters
Geometry from the node alone	1.49 GB	1.06 s	307,880
Geometry from node and neighbourhood	**1.57 GB**	**1.48 s**	**537,768**

Both with the filter of Section 1. Mean of 3 runs after a warm-up.

The summary costs +39% time, +5.4% memory and about 230,000 parameters. The parameters grow because every network that computes the geometry now reads a wider input.

Status. Whether the summary improves accuracy, and whether that is worth 39% more time, will be tested on the cluster: geometry from the node alone vs node and neighbourhood.

3. Further checks: the data pipeline and the path mechanism

3.1 The data pipeline matches the benchmark

Before changing the model, we checked that our data matches the HGB benchmark (ACM, DBLP, IMDB) exactly. We read the official files with HGB's own loader and compared them item by item:

- **Splits:** our test nodes are exactly HGB's test nodes. Our training and validation nodes together are exactly HGB's labelled set, with HGB's 20% held out for validation.
- **Labels:** all 3,025, 4,057 and 4,573 labels agree, including IMDB's multi-label vectors.
- **Edges:** our graph stores one edge per fact. Expanded back to both directions, it reproduces HGB's stored edge list exactly.

We could not reproduce HetSheaf's published rerun of R-GCN, for three reasons:

- their selected configuration came from a 150-run search that is not published;
- their paper states HGB's 24/6/70 split, but their code uses 27/3/70;
- their code duplicates every relation, giving ACM 14 relation types instead of 8 and twice the edges.

Their own pipeline also misses the published number with the configuration in their repository.

Decision. We use HGB's official split. Every baseline, HetSheaf included, will be rerun under our protocol. Published numbers will appear only in a clearly labelled note, not in the comparison tables.

3.2 A test of the path mechanism with a known answer

Our thesis is that a meta-path instance carries information through its intermediate node and the order of its relations, and that this information is lost when the instance is reduced to a relation between its two endpoints. Before running large comparisons, we wanted to see the mechanism do this on a graph where we know which instance holds the answer.

First design: the label is a co-paper's class, reached through the lead author

Each paper has a lead author and a second author, and each author has one other paper, a co-paper. The paper's label is the class of the co-paper reached through its lead author.

A model *without* path faces scored 0.996, so the task leaked. The cause was the neighbourhood summary. The lead author's summary already contains its co-paper's class, and the paper reads that summary through an ordinary edge. With the summary set to zero, the same trained model fell to chance (0.488). This is the overlap described in Section 2.

Second design: only the whole instance determines the label

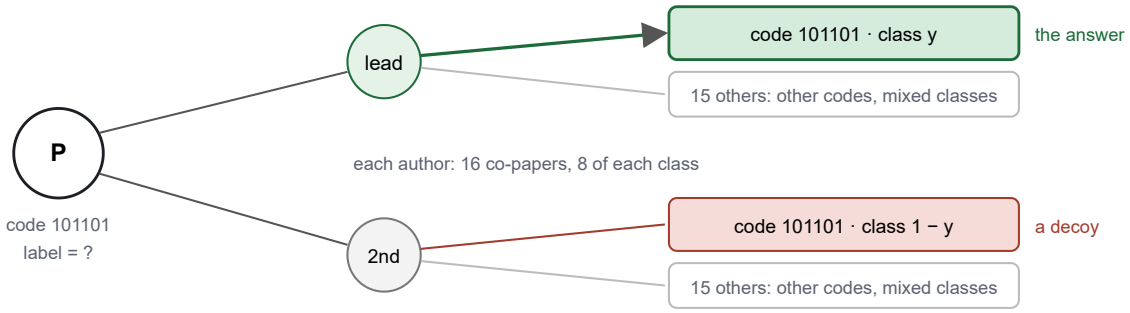


Figure 3. The second test. P's label is the class of the co-paper that is reached through the **lead** author **and** carries P's code. The second author also has a co-paper with P's code, of the opposite class. Neither the author alone, nor the code alone, nor the two endpoints alone give the answer; only the whole instance P – lead author – co-paper does.

The graph has 17,000 papers, 2,000 authors and 272,000 path instances. Each paper carries a 6-bit code, and each author has 16 co-papers, balanced by class. Of the 1,000 labelled papers, 500 are used for training. We fixed the pass criteria before any run:

- the full model must reach at least 0.90 on every seed;
- every control must stay at or below 0.60 (chance is 0.5).

Model (3 seeds)	Test accuracy	Reading
Controls: edges only (no path faces), endpoint relations only, own features only	0.48 – 0.56	at chance: the task does not leak
Full model, path weights learned	**0.50 / 0.58 / 0.54**	**at chance: fails the criterion**
Path faces with equal weights	0.58 / 0.56 / 0.53	at chance, as expected
Full model, correct path weights given (diagnostic)	1.00 / 0.996 / 1.00	solved

What the result shows

- **The rest of the model is not the bottleneck.** When we set the path weights to the correct answer and train everything else as usual, the model solves the task. The transports along the path, the filter and the classifier carry the information.
- **Learning the path weight is the bottleneck.**
  - The learned weights did separate lead-author instances from second-author ones. That depends on a property of a single node.
  - They never separated instances whose codes match from those whose codes do not. That needs a comparison between two nodes on the path.
  - The weight network sees all three nodes and has a hidden layer, so it can represent this rule in principle. It did not learn it from 500 papers in 200 epochs at default settings.
  - Its weights did not differ between training and test papers, so it was not memorising instances.

Status. The test neither confirms nor refutes the thesis on real data. The task is deliberately hard (an exact 6-bit match learned from 500 examples), and we did not tune for it, because tuning on a known-answer task would make a pass meaningless. It does show where the risk sits. Path instances add information beyond graph diffusion only when they are weighted by their content, and part of that content is a relationship between nodes on the path.

The weight function's ability to learn such comparisons is therefore central to the thesis. We carry this question, and both test tasks, into the next stage.

4. Cost

ACM, d = 32, one training step	Peak GPU memory	Time	Parameters
Submitted model	does not fit in 8 GB	—	1.62M
After lighter maps and direct path coupling (previous report)	2.5 GB	1.77 s	0.31M
After fact storage, path sampling and gates (previous report)	1.49 GB	1.06 s	0.30M
+ frequency-selective filter (Section 1)	1.49 GB	1.06 s	0.31M
+ neighbourhood summary (Section 2): current model	**1.57 GB**	**1.48 s**	**0.54M**

Peak GPU memory and time for one training step (forward and backward). The last three rows come from one measurement session (3 runs after a warm-up), the second row from the earlier session (5 runs).

	Previous model	Current model
Propagation	K diffusion steps: $O(K \cdot (M + F \cdot L) \cdot d^2)$	same K applications of A, plus channel mixing $O(N \cdot K \cdot d^2)$
Neighbourhood summary	—	one pass over the facts: $O(M \cdot d)$
Input to the geometry networks	d numbers per node	grows with the number of relation types; the first layer is evaluated once per node, not once per edge

N = number of nodes, M = number of facts, F = number of path instances, L = path length, K = filter degree.

The current model fits where the submitted one did not fit at all, and has about three times fewer parameters. On this GPU the memory saving can only be bounded from below, at 4.8× or more. Confirming the 10× target of the revision plan needs a measurement of the submitted model on a larger GPU. Almost all of this stage's added cost comes from the neighbourhood summary, not from the filter. Neither change alters how cost grows with graph size.

5. Next steps

1. **The new path mechanism.** It samples walks without a fixed meta-path list, gives each instance a learned weight, and reports meta-path importance afterwards instead of selecting meta-paths in advance. Before building it we will decide whether the weight function needs an explicit comparison between nodes on the path. Both known-answer tasks will be rerun on it. The first will run with the neighbourhood summary switched off, so that it tests the path weights rather than the summary.
2. **Comparisons on the cluster.** Each is fixed before running and uses the same tuning budget, with 10 seeds and paired tests:
   - the filter vs the old update, on the heterophilic datasets;
   - geometry with vs without the neighbourhood summary;
   - learned path weights vs endpoint-only weights vs no path faces, with the summary on in every arm.
3. **All baselines rerun** under the same protocol and split.

CHAPTER 3

9 November 2026

Heterophily of the evaluation datasets

The two heterophily measures, a check against a published value, one table for all node-classification datasets

H2S-NN Revision: Heterophily of the Evaluation Datasets

How heterophilic is each dataset we evaluate on, measured the way the source benchmarks define it

9 November 2026 Computed on CPU with the benchmarks' own published code

Why this chapter

The revised paper relates the model's gains to how heterophilic a dataset is. That comparison is only meaningful if every dataset is measured the same way. We therefore fixed two published measures and computed them with the reference code released by their authors, instead of writing our own versions. Before computing any new value, we checked that this pipeline reproduces a published number.

1. What the two measures capture

Both measures look at pairs of labelled nodes of the target type, for example two papers, that are linked through a **meta-path**: a short typed route such as paper–author–paper ("two papers share an author"). **MLH** (meta-path label homophily) is the share of such pairs whose two nodes carry the same label, averaged over all meta-paths of length two that start and end at the target type. We report it as *heterophily*, one minus that share. Higher means connected nodes disagree more often. The $\mathscr{H}^2$ **index** asks the same question but first subtracts the agreement that class sizes alone would produce if links ignored labels, and rescales the result (adjusted heterophily). A value near 0 means homophilic; near or above 1 means heterophilic. We use $\mathscr{H}^2$ as the primary measure because MLH can look homophilic merely because one class dominates.

A small example

Four papers P1–P4 have topics A, A, B, A. Two authors wrote them: author X wrote P1 and P2, author Y wrote P2, P3 and P4. Through "shares an author", including each paper with itself as the reference code does, there are 12 ordered paper pairs, and 8 of them agree on the topic. Agreement is 8/12 = 0.67, so MLH heterophily is 0.33, which looks homophilic. But topic A covers three of the four papers. When each paper is weighted by how many pairs it appears in, A accounts for 9 of 12 pair-ends and B for 3. Random pairing would agree with probability $(9/12)^2 + (3/12)^2 = 0.625$. The observed 0.67 is barely above that chance level: the adjusted homophily is $(0.67 - 0.625)/(1 - 0.625) = 0.11$, so $\mathscr{H}^2$ for this meta-path is 0.89, i.e. clearly heterophilic. The same links look homophilic to MLH and heterophilic to $\mathscr{H}^2$, because most of the agreement comes from class imbalance.

The values in the example are computed by hand. The reference code reproduces them exactly in our tests.

2. Check against a published value

Hetero²Net (Li et al., TPAMI 2025), which introduced MLH, reports for DBLP a homophily of 0.87 over its meta-path author–paper–author. Running the released code of $\mathscr{H}^2$GB (Lin et al., KDD 2025), which defines $\mathscr{H}^2$, on the same public DBLP graph gives **0.872**. We had fixed a tolerance of ±0.01 before running. The match depends on a detail: each author also counts as paired with itself, as in the released code. Without these self-pairs the value would be 0.80. The same pipeline also reproduces the published ACM value (0.60; ours 0.596) and IMDB value (0.24; ours 0.237). That paper's own ablation table lists these two values with the datasets swapped. Its main table and our results agree with each other.

3. Heterophily of the node-classification datasets

Dataset	MLH (heterophily)	$\mathscr{H}^2$ index	Source
DBLP	0.128	0.173	Computed with the reference code
ACM	0.262	0.399	Computed with the reference code
IMDB	0.325	0.737	Computed with the reference code, per genre (see note)
Freebase	0.846	0.995	Computed with the reference code, unlabelled books as the reference code treats them (see note)
ogbn-mag	0.873	0.877	Published ($\mathscr{H}^2$GB, Table 2)
mag-year	0.772	0.965	Published ($\mathscr{H}^2$GB, Table 2)
IEEE-CIS-G	0.135	0.985	Published ($\mathscr{H}^2$GB, Table 2)
H-Pokec	0.392	0.949	Published ($\mathscr{H}^2$GB, Table 2)

Both measures are in heterophily form (higher = more heterophilic), averaged over all length-2 meta-paths from the target type back to itself, as the benchmark that defines $\mathscr{H}^2$ does. For ACM this set includes paths through citations. The three-path set of the MLH paper (section 2) gives a different value. IEEE-CIS-G shows why $\mathscr{H}^2$ is the primary measure: MLH 0.135 looks homophilic, $\mathscr{H}^2$ 0.985 does not.

Notes.

- **IMDB (multi-label).** A movie can have several of five genres. We treat each genre as its own yes/no label, as the benchmark that released IMDB does when it classifies movies. Both measures are computed per genre with the reference code and then averaged over the five genres. The whole-set comparison of section 2 (two movies agree only when their genre sets are identical) gives 0.763 and serves only as the check against the published value. The 359 of 4,932 movies without genres receive random genre labels, as the unlabelled books of Freebase do (next note). Five different random labellings change the values by less than 0.003.
- **Freebase.** Only one book in five carries a label. As in the reference code, every unlabelled book receives a random label, so our values are measured the same way as the published ones in this table. The graph is made two-way, as the benchmark's own loaders do. The random labels dominate the result. Counting only pairs of two labelled books gives an $\mathscr{H}^2$ index of 0.882 and an MLH of 0.478. That figure is a diagnostic, not the reported value. Five different random labellings change the $\mathscr{H}^2$ index by about 0.001.

4. Datasets without node labels: not measurable

Both measures compare the labels of linked nodes, so they need node labels.

- **LastFM** (link prediction): users, artists and tags carry no labels, so the measures are not measurable.
- **PubMed** (link prediction): the public release has no labels. The disease labels exist only in a separate release that is no longer publicly downloadable, so the measures are not measurable at present.
- **Graph-classification datasets**: labels belong to whole graphs, not to nodes, so node-level heterophily does not apply. These datasets serve only to show that the model covers the task.